LAB 3: OpenMP

Embedded Processing Architectures

Zhang Ruida and Hugo Bueno

April 15th, 2024

Parallelization Implementation

The parallelization algorithm of this lab **has the same strategy to the one of lab 2, whereas OpenMP is used instead of Pthread**. Openmp also optimizes matrix operations through parallelization, but it is more concise than Pthread. The number of parallel regions can be controlled through instructions, thereby reducing the running time and achieving a more suitable use.

Results

Similarly to lab 2, Figures 1-5 showing the execution time vs the number of threads are presented for every one of the matrix sizes.

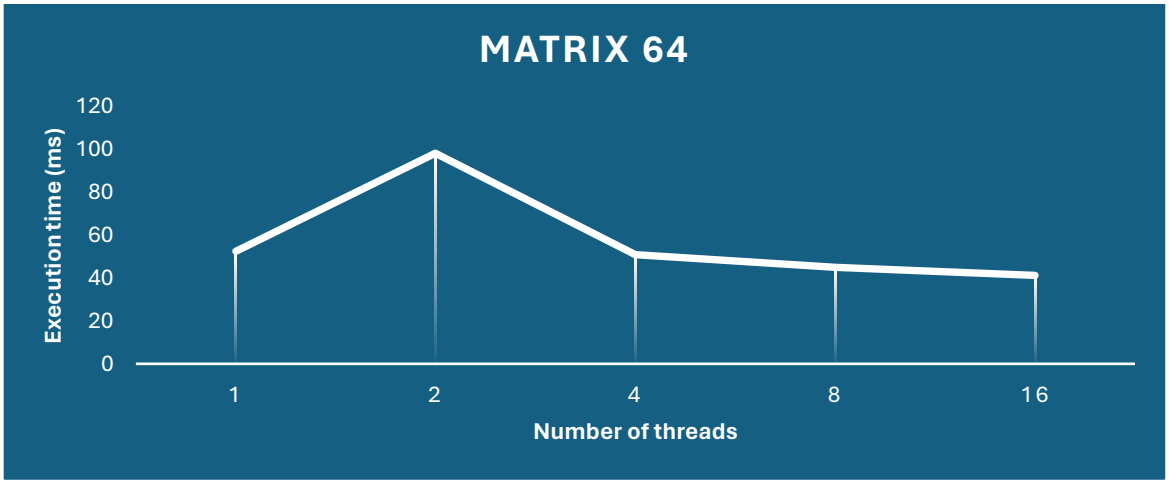

Figure 1. Execution time of the algorithm for different number of threads (64x64 matrix)

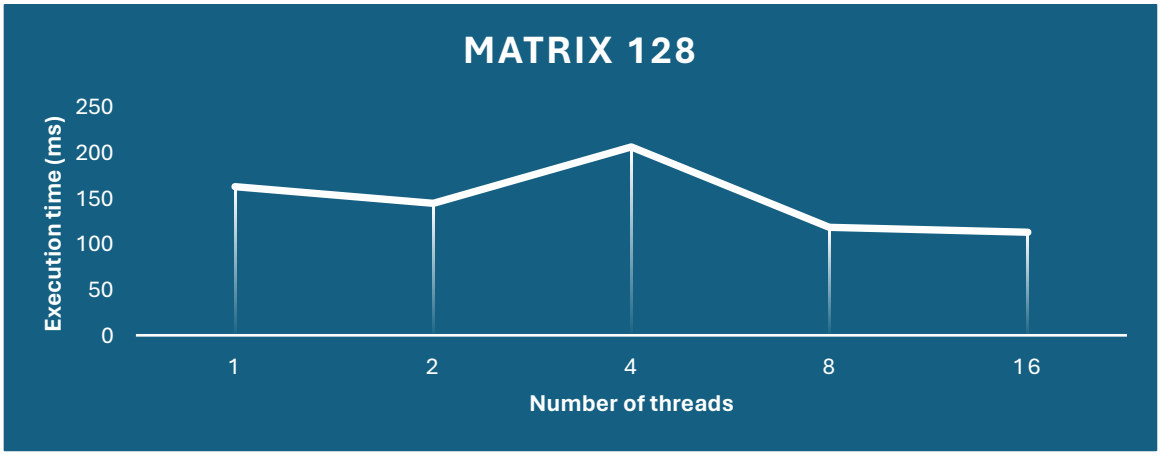

Figure 2. Execution time of the algorithm for different number of threads (128x128 matrix)

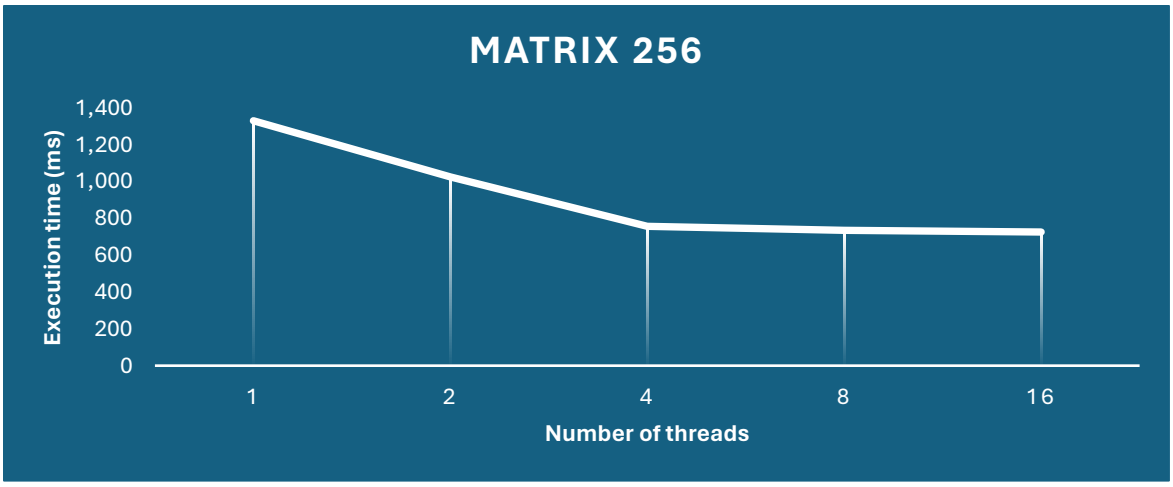

Figure 3. Execution time of the algorithm for different number of threads (256x256 matrix)

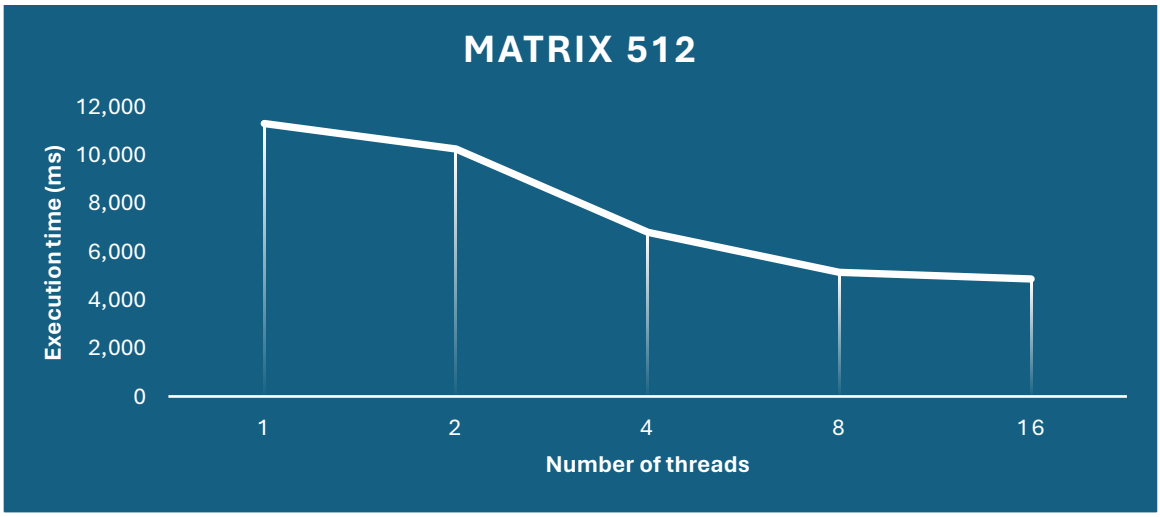

Figure 4. Execution time of the algorithm for different number of threads (512x512 matrix)

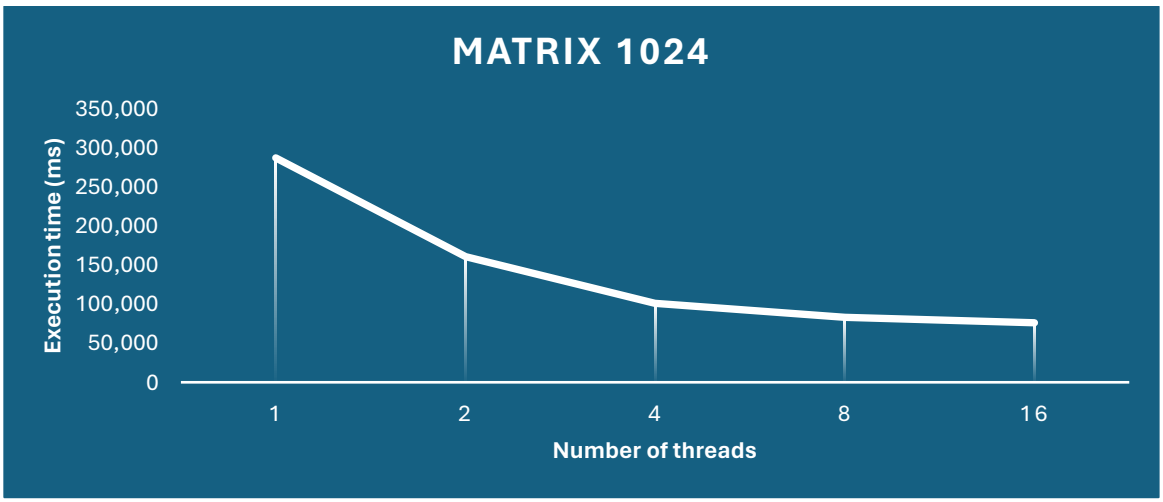

Figure 5. Execution time of the algorithm for different number of threads (1024x1024 matrix)

The exponential decrease of the execution of time can be seen again due to the duplication of the number of processes carried out in parallel in most of the cases. This limitation is, like in lab 2, the used SoC containing 8 cores which can handle multiple tasks in parallel. **However, while using OpenMP there are certain unexpected peaks detected** for low matrix sizes, when passing from 1-thread implementation to 2-thread implementation (e.g. Figure 1).

At the moment we do not know the real reason for this phenomenon happening, **but we believe it may have something to do with the balance of the time that it takes to create the new threads and the time of execution**. More concretely: for small matrix sizes which take relatively a short time to be executed even in series, the time of creating the new threads is not worth the improvement in performance that it delivers.

In conclusion, this only applies for small matrix sizes (because they are executed relatively fast) and for small number of threads which is not only one (because the improvement in performance is not enough).

Again, we can observe in Figure 6 that the absolute value for higher matrix sizes is on a higher order of magnitude. Therefore, the parallelization implementation is especially relevant for higher matrix sizes.

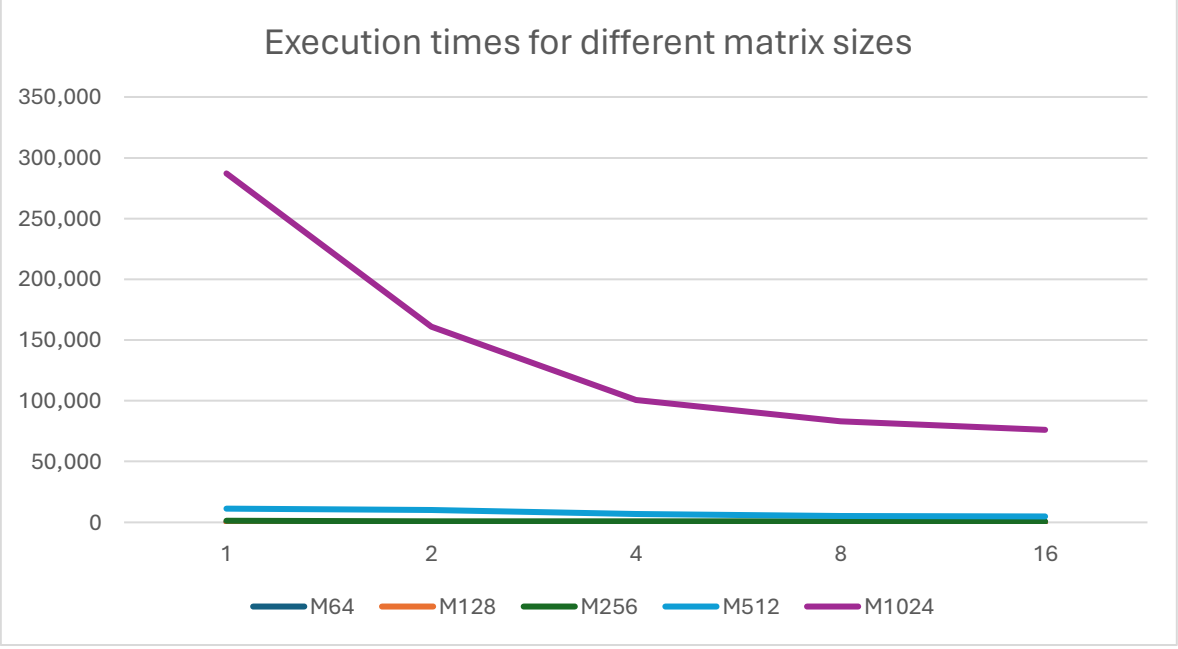

Figure 6. Execution times for different matrix sizes (ms)

Pthreads VS OpenMP

While both Pthreads and OpenMP may be used for this parallelization strategy, they have some differences that make them more suitable depending on the situation:

- OpenMP provides a set of easy-to-use compilation instructions that **make it easier to parallelize programs without having to manually manage thread creation and synchronization.** The tasks in the parallel area can be automatically assigned to have different threads for execution and optimized for different hardware platforms, so they can generally achieve a high performance. **However, the parallelization control it provides is relatively simple** and may not provide enough flexibility for some complex parallel modes.
- On the other hand, Pthreads provides more flexibility and control, **being suitable for scenarios that require higher concurrency details, whereas have higher coding complexity.**